Gradient Descent (GD): Theory, Variants, and Practical Guidance

A two-column reference with formulas, tables, and visual intuition

This document explains gradient descent from first principles (derivatives and gradients) to modern variants used in machine learning. It compares Batch, Stochastic, and Mini-batch gradient descent; discusses step-size choices, convergence intuition, common pitfalls, and widely used optimizers (Momentum, Nesterov, AdaGrad, RMSProp, Adam). Figures included are the images provided in the prompt.

Autor: generated by gpt 5.2

February 2026

1. What is Gradient Descent?

Gradient descent is a family of iterative algorithms for minimizing a function. In machine learning, the function is typically a loss (or cost) that measures how well a model with parameters θ fits data.

The core idea is simple: measure how the loss changes when θ changes (the gradient), then move θ a small step in the direction that decreases the loss the most.

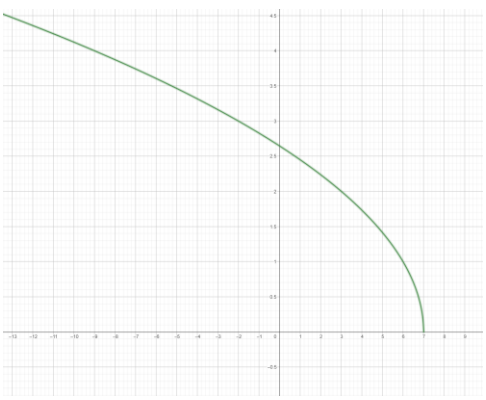

Fig. 9. Domain and function surface view: optimization moves over the surface J(θ) within its domain.

We write an optimization problem as:

minimize J(θ) over θ ∈ ℝ^d.

Here J is the objective (loss), θ is a vector of parameters, and d is the number of parameters (features, weights, biases, etc.).

- **Symbols**: θ (parameters), J(θ) (objective/loss), ∇J(θ) (gradient), α (learning rate), t (iteration index), B (mini-batch), n (dataset size).
- **Goal**: find θ that minimizes J(θ), often approximately, within a compute/time budget.

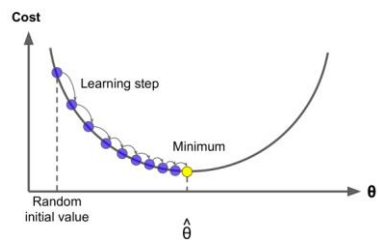

Fig. 8. Visual intuition for a gradient in 2D/3D: the gradient is perpendicular to level sets and points toward increasing values.

$$\min_{\theta \in \mathbb{R}^d} J(\theta)$$

Eq. 1. Generic optimization objective.

Why does the gradient point "uphill"? For a differentiable scalar function J(θ), the gradient ∇J(θ) is the direction of steepest increase. Therefore, −∇J(θ) is the direction of steepest decrease.

Gradient descent repeatedly applies small downhill steps until improvements become negligible or a stopping criterion is met.

2. Mathematical Foundations: Derivatives, Gradients, and Local Linearization

2.1 Derivative (1D) vs Gradient (dD)

In one dimension (a function J(θ) with a scalar θ), the derivative J′(θ) is the slope of the curve. If the slope is positive, moving left decreases J; if the slope is negative, moving right decreases J.

In multiple dimensions, the slope generalizes to the gradient vector.

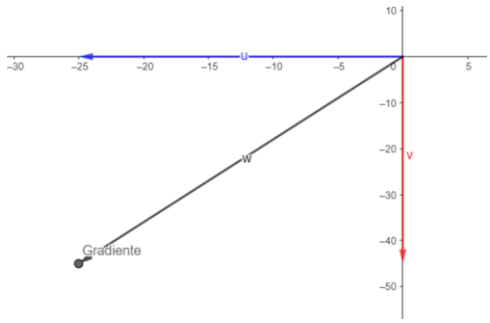

Fig. 7. Gradient as a direction in space: the gradient vector indicates steepest ascent; descent uses the negative gradient.

$$\nabla J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_d} \end{bmatrix}$$

Eq. 2. Gradient of a scalar objective with respect to a parameter vector.

2.2 Why the Update Works (Taylor Approximation)

A standard way to justify the update is a first-order Taylor approximation. For a small parameter change Δθ:

$$J(\theta + \Delta\theta) \approx J(\theta) + \nabla J(\theta)^{\mathsf{T}} \Delta\theta.$$

To decrease J as much as possible subject to a fixed step length, choose Δθ proportional to −∇J(θ).

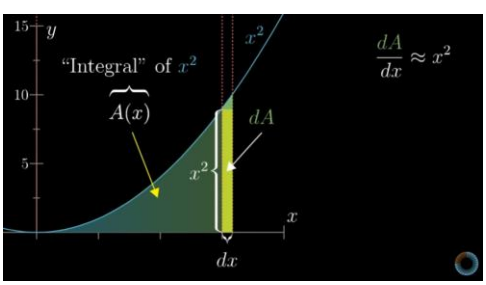

Fig. 10. Derivative as instantaneous rate of change: the slope at a point is the 1D precursor to gradients.

$$J(\theta + \Delta\theta) \approx J(\theta) + \nabla J(\theta)^{\mathsf{T}} \Delta\theta$$

Eq. 3. First-order Taylor approximation around \(\theta\).

3. The Core Algorithm

The basic gradient descent update (constant learning rate α > 0) is:

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t).$$

α is the learning rate (step size). Too small makes learning slow; too large can cause divergence or oscillations.

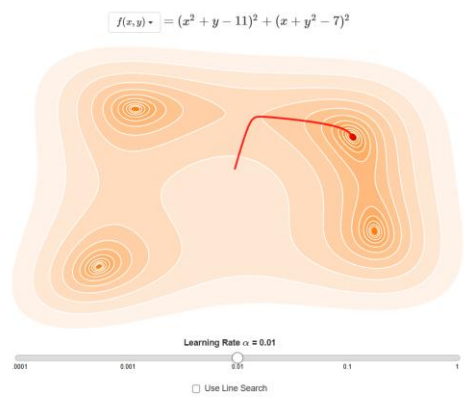

Fig. 1. Gradient descent on a 1D cost curve: repeated downhill steps toward a minimum (learning step shown).

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t)$$

Eq. 4. Basic gradient descent update with learning rate \(\alpha\).

Stopping criteria commonly include:

(1) small gradient norm (‖∇J(θ_t)‖ below a threshold),
(2) small improvement in J,
(3) maximum number of iterations,

(4) early stopping based on validation performance (common in ML).

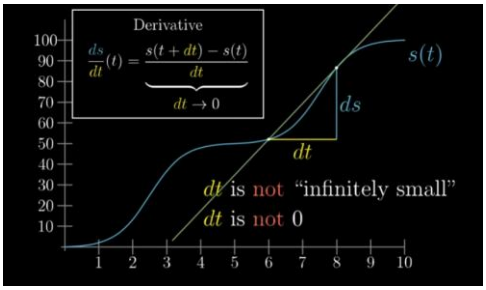

Fig. 11. First-derivative criterion: at a (local) optimum, the first derivative is zero (and the sign change indicates min/max).

3.1 Objective Functions in ML (Examples)

Example (linear regression). With data (x_i, y_i) and prediction ŷ_i = x_i^{T}θ, a common objective is mean squared error:

J(θ) = (1/2n) ∑_{i=1}^n (x_i^{T}θ − y_i)^2.

The gradient is ∇J(θ) = (1/n) X^T(Xθ − y), where X stacks the x_i row-wise.

$$J(\theta) = \frac{1}{2n}\sum_{i=1}^{n}(x_i^\top\theta - y_i)^2$$

Eq. 5. Mean squared error objective for linear regression (with \(1/2\) for convenience).

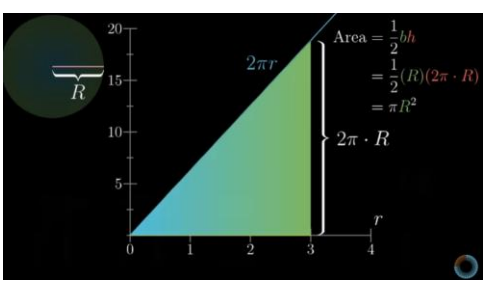

Fig. 14. Area/integral visual: connects derivatives, accumulation, and local linear reasoning used in optimization intuition.

$$\nabla J(\theta) = \frac{1}{n}X^\top(X\theta - y)$$

Eq. 6. Gradient of the linear regression MSE objective.

Example (logistic regression). For binary labels y_i ∈ {0,1} and σ(z)=1/(1+e^{−z}), with p_i = σ(x_i^{T}θ), the negative log-likelihood objective is:

J(θ)= −(1/n)∑[y_i log p_i + (1−y_i) log(1−p_i)].

Its gradient is ∇J(θ) = (1/n) X^T(p − y).

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Eq. 7. Logistic (sigmoid) function.

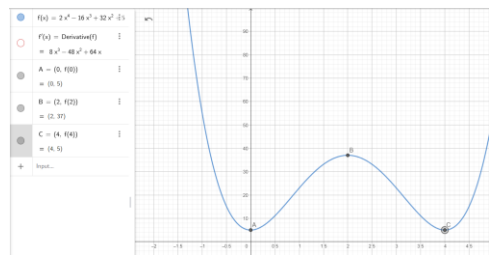

Fig. 12. Using the sign of the first derivative to detect increasing/decreasing regions; helpful for understanding descent directions.

$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log p_i + (1-y_i)\log(1-p_i)], p_i = \sigma(x_i^\top\theta)$$

Eq. 8. Logistic regression negative log-likelihood (cross-entropy).

4. Learning Rate and Step-Size Control

$$\nabla J(\theta) = \frac{1}{n}X^\top(p-y)$$

Eq. 9. Gradient for logistic regression cross-entropy.

Learning rate selection is often the most important practical choice. If α is too large, updates can overshoot the minimum; if too small, training can take

an impractically long time.

A helpful mental model: α scales how much you trust the local slope information from the gradient.

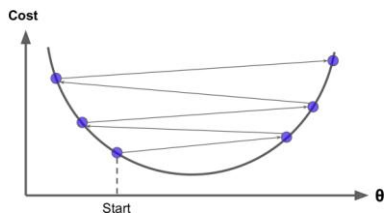

Fig. 2. Overshooting with large steps can cause bouncing across the valley; controlling the learning rate stabilizes descent.

Common learning-rate schedules:

- Constant α: simplest; can work with careful tuning.
- Step decay: reduce α by a factor at certain epochs.
- Exponential decay: $\alpha_t = \alpha_0 \gamma^t$.
- 1/t decay: $\alpha_t = \alpha_0 / (1 + kt)$.

In deep learning, warmup + decay schedules are also widely used.

5. Types of Gradient Descent

The three classic "types" of gradient descent differ in how much data they use to estimate the gradient.

Let $g(\theta) = \nabla J(\theta)$ be the full gradient and let $g_B(\theta)$ be an estimate computed on a subset (mini-batch) B of the data.

5.1 Batch vs SGD vs Mini-batch

Batch Gradient Descent computes the gradient using all n training examples each iteration. This yields a stable direction but can be expensive when n is large.

Stochastic Gradient Descent (SGD) uses a single example (or very small batch) per update. It is noisy but fast and can escape shallow local traps.

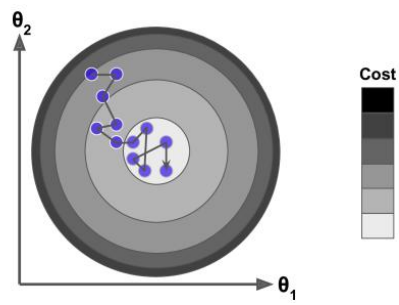

Fig. 4. Stochastic gradient descent trajectory illustrates noisy but fast progress toward a good region.

Mini-batch Gradient Descent is the workhorse of modern ML: each update uses a small batch (e.g., 32–4096 examples). It reduces noise compared to pure SGD while leveraging efficient matrix operations on modern hardware.

The mini-batch size also affects the "noise level" and generalization behavior.

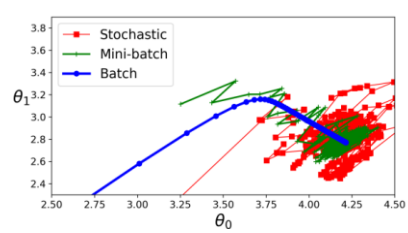

Fig. 3. Path comparison: Batch (smooth), Mini-batch (moderately noisy), and Stochastic (highly noisy) updates in parameter space.

A compact way to write the three variants (for empirical risk $J(\theta)=(1/n)\sum \ell_i(\theta)$):

Batch: $g_t = (1/n)\sum_{i=1}^{n} \nabla \ell_i(\theta_t)$
SGD: $g_t = \nabla \ell_{i_t}(\theta_t)$
Mini-batch: $g_t = (1/|B_t|)\sum_{i \in B_t} \nabla \ell_i(\theta_t)$

Update: $\theta_{t+1}=\theta_t - \alpha g_t$.

Table 1. Comparison of Batch, Stochastic, and Mini-batch Gradient Descent.

Variant	Gradient used per update	Compute per update	Trajectory behavior	Typical use cases
Batch GD	All n examples	High	Smooth, deterministic	Small/medium datasets; convex problems; when exact gradients matter
Stochastic GD (SGD)	1 example	Very low	Noisy; can wander but explores	Huge datasets; online learning; sometimes better at escaping saddle points
Mini-batch GD	A batch (e.g., 32–4096)	Medium	Moderately noisy; efficient on GPUs/TPUs	Default choice for deep learning and large-scale ML

6. Beyond Basic GD: Momentum and Adaptive Optimizers

Momentum accelerates gradient descent by combining the current gradient with an exponentially decayed moving average of past gradients. This helps traverse narrow valleys and reduces oscillations.

Intuition: it behaves like a heavy ball rolling downhill, accumulating velocity in consistent directions.

$$v_{t+1} = \beta v_t + \nabla J(\theta_t), \theta_{t+1} = \theta_t - \alpha v_{t+1}$$

Eq. 10. Momentum method (one common formulation).

6.1 Momentum and Nesterov Accelerated Gradient

Nesterov momentum (NAG) is a "lookahead" version of momentum. It computes the gradient after moving in the direction of the current velocity, which can yield better correction and stability in practice.

Both Momentum and NAG are popular in convex optimization and deep learning.

$$v_{t+1} = \beta v_t + \nabla J(\theta_t - \alpha\beta v_t), \theta_{t+1} = \theta_t - \alpha v_{t+1}$$

Eq. 11. Nesterov accelerated gradient (lookahead).

6.2 AdaGrad, RMSProp, and Adam

Adaptive methods change the effective learning rate per-parameter. They are helpful when gradients have different scales across coordinates.

AdaGrad accumulates squared gradients and decreases the step size over time; RMSProp uses an exponential moving average to avoid AdaGrad's overly aggressive decay.

$$g_t = \nabla J(\theta_t), s_t = \rho s_{t-1} + (1-\rho) g_t \odot g_t, \theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{s_t} + \varepsilon} \odot g_t$$

Eq. 12. RMSProp-style adaptive scaling (element-wise).

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t \odot g_t, \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

Eq. 13. Adam optimizer update (element-wise).

Adam combines momentum (first moment) and RMSProp-like normalization (second moment), plus bias correction. It is widely used because it often works "out of the box," though good results still require tuning.

A common practical approach: start with Adam, then validate with SGD+Momentum for final performance in some tasks.

Table 2. Popular extensions and optimizers related to gradient descent.

Method	Update idea (high-level)	Strengths	Common downsides
Momentum	Velocity accumulates gradients	Faster in valleys; less zig-zag	Extra hyperparameter; can overshoot if α too large
Nesterov (NAG)	Lookahead gradient with momentum	Often more stable than momentum	Slightly more complex; tuning still needed
AdaGrad	Per-parameter step shrinks with accumulated squared gradients	Good for sparse features	Learning rate can decay too much over long training
RMSProp	EMA of squared gradients normalizes steps	Handles nonstationary objectives	Needs tuning; not always best for convex problems
Adam	Momentum + RMSProp + bias correction	Strong default; fast early progress	May generalize worse in some settings; can need decay/weight decay tuning

7. Convergence Intuition and Geometry

Geometry matters. In "round" level sets, gradient descent moves smoothly to the minimum. In "elongated" level sets (ill-conditioned problems), steps can zig-zag because the gradient points across the narrow direction of the valley.

Preconditioning (feature scaling, normalization, or second-order ideas) can dramatically improve convergence.

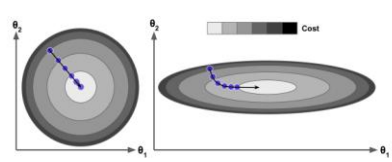

Fig. 5. Gradient descent in two parameters (θ0, θ1): level sets and a curved path toward the minimum (illustrates conditioning and curvature).

In non-convex landscapes (typical in deep learning), there may be many local minima, saddle points, and plateaus. Gradient descent methods still work well in practice, but behavior depends strongly on initialization, learning rate, and noise.

Noise from SGD/mini-batches can help escape saddle points and flat regions.

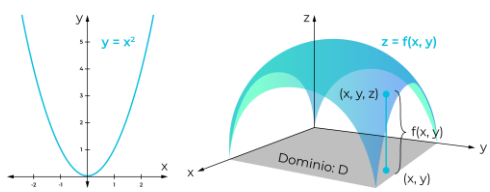

Fig. 13. Non-convex contour example with a learning-rate control: illustrates sensitivity to step size and the possibility of different basins.

8. Pitfalls, Diagnostics, and Best Practices

A useful diagnostic is to monitor:

- training loss vs. iteration,
- validation loss vs. iteration,
- gradient norms,
- parameter update norms,
- learning rate over time.

Divergence, oscillation, or a loss that becomes NaN often signals an overly large learning rate or numerical instability.

- **Rule of thumb**: if the loss increases sharply or becomes NaN, α is likely too large.
- **Stability tools**: learning-rate decay, gradient clipping, normalization, and momentum.
- **Signal vs noise**: smaller batches add gradient noise; larger batches make updates smoother but can be slower per update.

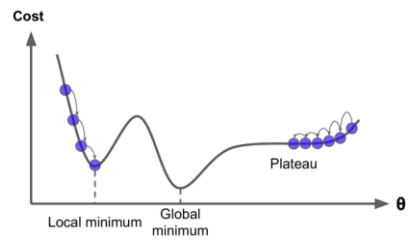

Fig. 6. Common pitfalls: local minima, global minima, and plateaus; noisy updates may help traverse flat regions.

Common pitfalls and how to fix them:

• Overshooting/divergence: reduce α, add decay, use gradient clipping.
• Slow learning: increase α slightly, use momentum, normalize inputs.
• Getting stuck on plateaus: use momentum/Adam, adjust α schedule.
• Strong zig-zagging: feature scaling, momentum, smaller α.

Always change one factor at a time to keep debugging clear.

Table 3. Troubleshooting common gradient descent behaviors.

Symptom	Likely cause	Fix
Loss explodes or becomes NaN	Learning rate too large; numerical instability	Lower α; add decay; gradient clipping; check data scaling
Loss decreases very slowly	α too small; poor conditioning	Increase α; normalize features; add momentum

Oscillation around minimum	α too large; ill-conditioned valley	Lower α; add momentum; consider schedule
Stuck on plateaus	Small gradients; saddle region	Use momentum/Adam; adjust α; consider restarts
Zig-zagging path	Features on different scales	Standardize features; use momentum; use adaptive optimizer

Practical checklist (typical ML workflow):

1) Normalize/standardize features.
2) Pick an optimizer (mini-batch SGD+Momentum or Adam).
3) Choose an initial α and a schedule.
4) Use early stopping and validation curves.
5) Tune batch size, α, and regularization together.

When in doubt, start simple and add complexity only as needed.

1. Normalize input features (mean 0, variance 1) when possible.
2. Start with mini-batch size 32–256 (or larger if hardware allows).
3. Try Adam with α=1e−3 as a baseline; validate and tune.
4. If you need stronger generalization, also try SGD+Momentum (often with a smaller α and schedule).
5. Use a validation set and early stopping; keep logs of α, loss, and metrics.

9. Quick Summary

Summary: gradient descent updates parameters by moving against the gradient of the objective. Batch, stochastic, and mini-batch variants trade off compute cost, noise, and stability. Learning-rate choice and scaling/conditioning often matter more than the exact variant.

Modern optimizers (Momentum, RMSProp, Adam) can improve speed and robustness, but they do not replace careful monitoring and good data preprocessing.